

Лабораторная работа № 1 по курсу дискретного анализа: сортировка за линейное время

Выполнил студент группы М8О-203БВ-24 МАИ *Сылкин Илья*.

Условие

Требуется разработать программу, осуществляющую ввод пар «ключ-значение», их упорядочивание по возрастанию ключа указанным алгоритмом сортировки за линейное время и вывод отсортированной последовательности.

Вариант задания: тип ключа — дата в формате DD.MM.YYYY (например 1.1.1, 1.9.2009, 31.12.2009). Тип значения — строки переменной длины до 2048 символов. Метод сортировки — поразрядная сортировка (Radix Sort, LSD).

Метод решения

Поразрядная LSD-сортировка (Least Significant Digit) работает за $O(n \cdot k)$, где n — количество элементов, k — количество разрядов ключа. Для дат k фиксировано: максимум 4 цифры в годе, 2 в месяце, 2 в дне. Поэтому сложность фактически линейная — $O(n)$.

Алгоритм выполняет три серии проходов устойчивой сортировкой подсчётом: сначала по всем разрядам дня, затем месяца, затем года. Порядок серий — от наименее значимой части даты к наиболее значимой. Поскольку каждый проход стабилен, при равенстве годов сохраняется порядок по месяцу, при равенстве месяцев — по дню.

Чтобы не копировать строки значений при каждом проходе, в финальной реализации сортируются только целочисленные индексы. Сами строки хранятся в отдельном векторе `lines` и остаются неподвижными в памяти на протяжении всей работы программы.

Описание программы

Программа реализована в одном файле `main.cpp` на C++20.

Типы данных:

- `struct Date` — хранит три целых числа: `day`, `month`, `year`.
- `vector<string> lines` — оригинальные строки входного файла; не изменяются после чтения.
- `vector<Date> dates` — разобранные даты, параллельный массив к `lines`.

- `vector<int> order` — массив индексов, который непосредственно сортируется.
- `vector<int> buffer` — вспомогательный буфер для сортировки подсчётом; создаётся один раз.

Функции:

- `parse_date(s, d)` — разбирает строку формата D.M.YYYY в структуру `Date`.
- `counting_sort_digit(order, buffer, dates, exp, part)` — один проход устойчивой сортировки подсчётом по заданному разряду (`exp`) заданной части даты (`part`). В конце выполняет `order.swap(buffer)` за $O(1)$ — меняет указатели векторов без копирования элементов.
- `radix_sort(order, buffer, dates, part)` — поразрядная сортировка по одной части даты: находит максимальное значение, затем последовательно вызывает `counting_sort_digit` для каждого разряда.
- `main()` — читает входные данные, запускает три серии сортировки (день → месяц → год), выводит строки в отсортированном порядке по массиву индексов.

Дневник отладки

Посылки 157869812, 157876780, 157887759, 157892626 — WA на тестах 1 и 11. Первые реализации использовали `std::chrono` для перевода даты в количество дней от фиксированной точки отсчёта, после чего поразрядная сортировка применялась к этому единственному числовому ключу. Подход работал на всех самостоятельно составленных тестах, однако получал WA на тесте 11 системы. Предположительно, `std::chrono::year_month_day` некорректно обрабатывает даты с очень малыми годами (например 1.1.1) или даты, не прошедшие валидацию по григорианскому календарю. Было принято решение отказаться от `chrono` и перейти к сортировке непосредственно по трём числовым полям — дню, месяцу и году.

Следует отметить, что в ходе перехода на новый подход также были допущены ошибки в логике работы программы: неправильный порядок проходов сортировки (DAY → MONTH → YEAR вместо требуемого YEAR → MONTH → DAY) и нарушение стабильности из-за итерации вперёд при расстановке элементов. Все эти проблемы были исправлены до перехода к следующей серии посылок.

Посылки 158092352, 158092437 — MLE на тесте 13 (164 МБ при лимите 150 МБ). Класс `Date` хранил строки `date` и `key` вместе с числовыми полями. При каждом проходе сортировки создавался временный вектор

result_vector размером n , куда копировались объекты Date со всеми строками. Одновременно в памяти существовали исходный вектор и его копия — суммарный расход строк удваивался.

Посылки 158092520, 158092624 — MLE на тесте 13 (174-175 МБ). Попытка оптимизировать память: временный вектор вынесен из функции и передаётся как параметр buffer, чтобы не выделять память заново на каждом проходе. После swap добавлено явное обнуление буфера. Однако это не помогло, так как строки по-прежнему хранились внутри Date и копировались при каждом присваивании `buffer[i] = data[i]`.

Посылка 158093478 — ОК (74.9 МБ, 1.021 с). Строки полностью вынесены из класса Date в отдельный вектор lines. Структура Date сведена к трём числам (12 байт на элемент). Вместо сортировки вектора объектов сортируется вектор целочисленных индексов order (4 байта на элемент) — строки при этом не перемещаются вообще. Задача принята.

Тест производительности

Тестирование проводилось на случайно сгенерированных данных с датами в диапазоне 1.1.1 — 31.12.9999 и значениями длиной до 200 символов. Измерялось реальное (*wall clock*) время командой `time ./main < input > /dev/null`.

Количество строк	Время (сек)
100	0.008
1 000	0.017
10 000	0.136
100 000	0.978
1 000 000	8.981

На графике видно, что при увеличении числа строк в 10 раз время растёт примерно в 8–10 раз, что соответствует линейной сложности $O(n)$. Небольшое отклонение от идеальной прямой на малых объёмах объясняется фиксированными накладными расходами на запуск процесса и операции ввода-вывода.

Выводы

Поразрядная LSD-сортировка эффективна для сортировки данных с ключами фиксированной структуры — дат, целых чисел, строк ограниченной длины. Алгоритм работает за $O(n)$ при фиксированном числе разрядов, что делает его быстрее сравнительных сортировок ($O(n \log n)$) на больших объёмах.

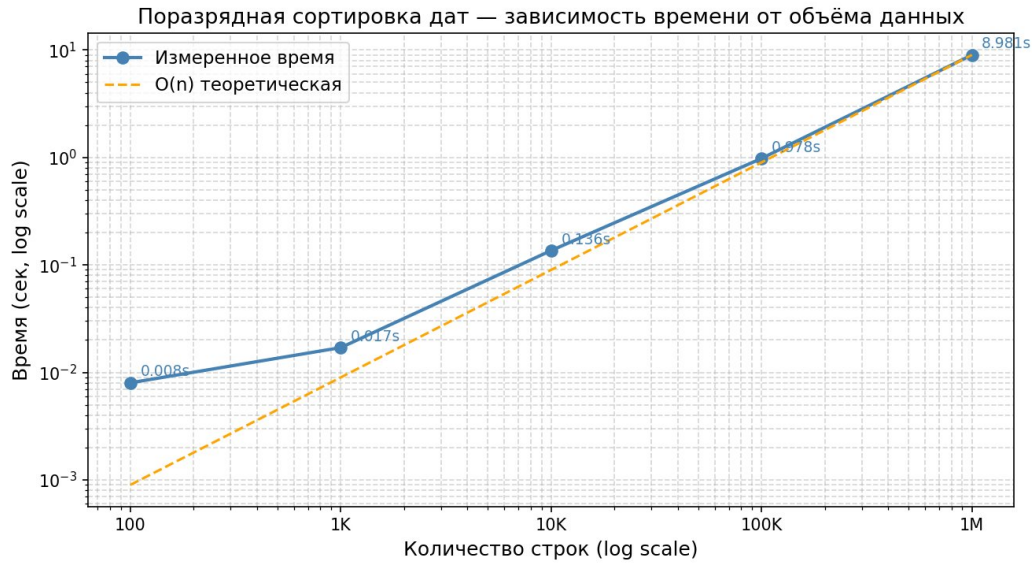


Рис. 1: Зависимость времени работы от объёма данных (логарифмический масштаб). Синяя линия — измеренное время, оранжевая пунктирная — теоретическая $O(n)$.

Основные трудности при реализации: обеспечить устойчивость каждого прохода (итерация с конца массива) и выбрать правильный порядок проходов — от наименее значимой части ключа к наиболее значимой. Вторая серьёзная практическая проблема — управление памятью: хранение строк непосредственно в сортируемых объектах приводит к их многократному копированию и превышению лимита памяти. Решение — хранить строки отдельно и сортировать только лёгкие числовые индексы.